

Efficient Particle Simulation with Spatial Hashing

การจำลอง Particle แบบมีประสิทธิภาพด้วย Spatial Hashing

นายอาหมิน เจ๊ะบือราเฮง | GitHub: Aftermin

Github Link: <https://github.com/Aftermin/AnimationProject.git>

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ จุฬาลงกรณ์มหาวิทยาลัย

วิชา 2110512 Computer Animation | เทอมปลาย ปีการศึกษา 2568

อาจารย์ผู้สอน: รศ.ดร.พิษณุ คนองชัยยศ

คำสำคัญ — Particle Simulation, Spatial Hashing, Collision Detection, Real-time Animation, Position-Based Dynamics

บทคัดย่อ — งานวิจัยนี้นำเสนอการพัฒนาและเปรียบเทียบอัลกอริทึมสำหรับการตรวจจับการชน (collision detection) ใน Particle System แบบ real-time บนระบบ 2 มิติ อัลกอริทึมพื้นฐาน (Baseline) ใช้วิธี Brute-force ซึ่งมี Time Complexity $O(N^2)$ ทำให้ประสิทธิภาพลดลงอย่างมากเมื่อจำนวน particle เพิ่มขึ้น วิธีที่เสนอ (Proposed) ใช้ Spatial Hashing เพื่อแบ่งพื้นที่ออกเป็น grid และตรวจสอบเฉพาะ particle ใน 3×3 neighborhood ลด complexity เหลือ $O(N)$ โดยเฉลี่ย ผลการทดลองที่ $N=10,000$ แสดงให้เห็นว่า Proposed เร็วกว่า Baseline ประมาณ 77 เท่า นอกจากนี้ยังเพิ่มความสมจริงด้วย gravity, restitution, trail และ specular highlight

1. นิยามปัญหาและที่มา

Particle System เป็นเทคนิคสำคัญใน Computer Animation ที่ใช้จำลองปรากฏการณ์ทางธรรมชาติ เช่น ฝน ควัน ไฟ และของเหลว การชนกัน (collision) ระหว่าง particle ต้องถูกตรวจสอบทุกเฟรม ใน real-time ซึ่งต้องการความเร็วสูงมาก วิธีพื้นฐานที่ง่ายที่สุดคือการตรวจสอบทุกคู่ (Brute-force) ซึ่งมี Time Complexity $O(N^2)$ ทำให้ไม่สามารถรองรับ particle จำนวนมากได้ เมื่อ $N=200$ ต้องตรวจ $\sim 20,000$ คู่ แต่เมื่อ $N=2,000$ ต้องตรวจเกือบ 2,000,000 คู่ ส่งผลให้ FPS ลดลงอย่างมาก

1.1 นิยามเชิงรูปนัย

- Input: ชุด particle $P = \{p_1, \dots, p_n\}$ แต่ละตัวมีตำแหน่ง (x, y) , velocity (v_x, v_y) และ radius r
- Output: ตำแหน่งและความเร็วที่อัปเดตหลัง collision ทุกเฟรม พร้อม FPS ≥ 30
- Constraints: N สูงสุด 10,000, real-time ≥ 30 FPS, numerical stability คงที่

1.2 ความท้าทาย

ปัญหาหลักคือ $O(N^2)$ ที่เพิ่มขึ้นแบบ quadratic เมื่อ N มากขึ้น นอกจากนี้ยังมี numerical instability เมื่อ particle overlap และต้องจัดการ edge cases เช่น particle ชนขอบพร้อมรักษา visual realism

2. งานที่เกี่ยวข้องและ Baseline

อัลกอริทึม Brute-force เป็นวิธีมาตรฐานที่สอนในชั้นเรียน ตรวจสอบทุกคู่ (i, j) ที่ $i < j$ ด้วย distance check และแก้การชนด้วย impulse-based response วิธีนี้ถูกต้อง 100% แต่ไม่ scalable

วิธี	Complexity	ข้อจำกัด
Brute-force (Baseline)	$O(N^2)$	ช้ามากเมื่อ $N > 500$
Spatial Hashing (Proposed)	$O(N)$	Worst case เข้าใกล้ $O(N^2)$
BVH	$O(N \log N)$	Rebuild overhead สูง
Grid (fixed)	$O(N)$	ไม่ยืดหยุ่นกับ size ต่างกัน

ตารางที่ 1: เปรียบเทียบอัลกอริทึม collision detection

3. วิธีที่เสนอ: Spatial Hashing

Spatial Hashing เป็นเทคนิค broad-phase collision detection ที่แบ่งพื้นที่ออกเป็น uniform grid แล้วจัดเก็บ particle ใน hash table โดยใช้ตำแหน่ง cell เป็น key ทำให้การค้นหา neighbor มี complexity $O(1)$ โดยเฉลี่ย

3.1 แนวคิดหลัก

แทนที่จะตรวจสอบทุกคู่ Spatial Hashing ใช้ความจริงที่ว่า particle สองตัวจะชนกันได้ ก็ต่อเมื่ออยู่ใกล้กันเท่านั้น กำหนด cell size = $2 \times \text{radius}$ เพื่อให้ particle ที่ชนกันได้อยู่ใน 3×3 neighborhood เสมอ

3.2 อัลกอริทึม

```
1. FUNCTION update_proposed(particles, w, h, hash):
2.   FOR p IN particles: p.update(w, h)
3.   hash.clear()
4.   FOR p IN particles: hash.insert(p)
5.   FOR p1 IN particles:
6.     neighbors = hash.get_nearby(p1)
7.     FOR p2 IN neighbors:
8.       IF id(p1) < id(p2):
9.         resolve_collision(p1, p2)
```

3.3 การวิเคราะห์เชิงทฤษฎี

- Time Complexity: $O(N)$ average — แต่ละ particle ตรวจสอบ neighbor ใน 9 cells worst case $O(N^2)$ เมื่อทุก particle อยู่ใน cell เดียวกัน
- Space Complexity: $O(N)$ — hash table มี entry ไม่เกิน N entries
- Stability: ใช้ inelastic collision (restitution=0.85) ทำให้ระบบสูญเสียพลังงานตามธรรมชาติ ลด jitter และ overlap artifact

4. รายละเอียดการพัฒนา

หัวข้อ	รายละเอียด
ภาษา / Framework	Python 3.13, pygame 2.6
โครงสร้างข้อมูล	dict[(col,row) → list[Particle]]
Collision response	Impulse-based, restitution=0.85
Physics	Gravity=0.15, wall bounce=0.82
Visual	Trail, glow, specular highlight
Platform	macOS, CPU only

ตารางที่ 2: รายละเอียด implementation

SpatialHash ใช้ Python dict โดยมี (col, row) เป็น key การ clear และ rebuild ทุกเฟรมใช้เวลา $O(N)$ ซึ่งเร็วกว่า BVH tree ที่ต้องใช้ $O(N \log N)$

4.1 การจัดการกรณีขอบ

เมื่อ particle ขนขอบจะคูณ velocity ด้วย restitution (0.82) และหยุด jitter เมื่อ $|v_y| < 0.8$ ป้องกัน particle สั่นที่พื้น positional correction แก้ overlap โดยดัน particle ออก 50% ต่อฝ่าย

5. การทดลอง

ทดสอบบน macOS, Apple M-series, RAM 16 GB วัด median runtime จาก 50 frames (warm-up 10 frames) ทำซ้ำ 3 รอบ ชุดข้อมูลสังเคราะห์: random positions, sizes 100–10,000

5.1 Scaling Test

N	Baseline (ms)	Proposed (ms)	Speedup
100	0.5996500010951420	0.15136250003706700	3.96x
500	14.820512499136400	0.8466667000902820	17.5x
1000	60.11975830042500	2.1772583000711200	27.61x
2000	244.54871250054600	5.654624999442600	43.24x
5000	1523.3015833000500	24.61817500006870	61.87x
10000	6185.053341700400	79.730862499855	77.57x

ตารางที่ 3: ผลการทดลอง scaling test

5.2 ตัวชี้วัด

- Runtime (ms/frame): median จาก 50 frames, `time.perf_counter()`
- Energy Drift (%): $|KE_{final} - KE_0| / KE_0 \times 100$ จาก 200 frames
- Memory (MB): peak memory วัดด้วย `tracemalloc`

6. ผลการทดลองและการวิเคราะห์

จากตารางที่ 3 Proposed มี runtime เพิ่มขึ้นแบบ near-linear ในขณะที่ Baseline เพิ่มขึ้นแบบ quadratic ที่ $N=2,000$ Proposed เร็วกว่าประมาณ 43.24 เท่า Energy drift ของ Proposed ต่ำกว่า Baseline เนื่องจาก inelastic collision ทำให้พลังงานในระบบลดลงอย่างสม่ำเสมอ ในขณะที่ Baseline ใช้ elastic collision ที่อาจเกิด artifact เมื่อ particle หนาแน่น ข้อจำกัดของ Proposed คือ worst case ยังเป็น $O(N^2)$ เมื่อ particle กองรวมกัน และ memory overhead จาก hash table



รูปที่ 1: เปรียบเทียบ runtime ระหว่าง Baseline $O(N^2)$ และ Proposed $O(N)$ เมื่อจำนวน particle เพิ่มขึ้น จะเห็นว่า Baseline เพิ่มแบบ quadratic ในขณะที่ Proposed เพิ่มแบบ near-linear

7. บทสรุปและงานต่อยอด

งานนี้แสดงให้เห็นว่า Spatial Hashing ลด complexity จาก $O(N^2)$ เหลือ $O(N)$ โดยเฉลี่ย ทำให้ simulation รองรับ particle ได้มากกว่า 5,000 ตัวที่ real-time บน CPU ทั่วไป การเพิ่ม gravity, restitution, trail และ specular highlight ทำให้ visual realism ดีขึ้น

งานต่อยอด: (1) GPU acceleration ด้วย Taichi หรือ CUDA (2) เพิ่ม varying radius particle (3) เปรียบเทียบกับ BVH และ kd-tree (4) เพิ่ม constraint solver สำหรับ cloth simulation

รายการอ้างอิง

- [1] M. Müller et al., "Position Based Dynamics," J. Visual Communication, 2007.
- [2] M. Teschner et al., "Optimized Spatial Hashing for Collision Detection of Deformable Objects," VMV, 2003.
- [3] E. Catto, "Continuous Collision," GDC, 2013.
- [4] pygame community, "pygame 2.6 Documentation," 2024. [Online] <https://www.pygame.org>
- [5] R. Bridson, Fluid Simulation for Computer Graphics, A K Peters, 2008.